

SESSION 02

JAVASCRIPT

Operators, Expressions & Logic

ARITHMETIC

+ - * / % **

ASSIGNMENT

= += -= *= /=

COMPARISON

== === != !== > <

LOGICAL + TERNARY

&& || ! ? :

Table of Contents

A page-by-page learning path. Every major concept begins on a fresh page.

PAGE	SECTION
03	Session Overview
04	Operators & Expressions
05	Arithmetic Operators
06	Basic Arithmetic Examples
07	Modulus Operator
08	Pre vs Post Increment
09	Pre-Increment
10	Post-Increment
11	Pre vs Post Decrement
12	Execution Trace
13	Compound Expressions
14	Assignment Operators
15	Assignment in Practice
16	Comparison Operators
17	Loose vs Strict Equality
18	Relational Operators
19	Inequality Operators
20	Logical Operators

PAGE	SECTION
21	Logical Truth Tables
22	Logical Operators in Practice
23	Short-Circuit Evaluation
24	Ternary Operator
25	Ternary vs if-else
26	Practice 1 - Positive / Negative
27	Practice 2 - Largest Number
28	Practice 3 - Even / Odd
29	Additional Practice - 4 to 8
30	Additional Practice - 9 to 13
31	Challenge Questions
32	Answer Key - 1 to 5
33	Answer Key - 6 to 13
34	Challenge Answers
35	Interview Revision - Core
36	Interview Revision - Applied
37	Quick Revision Sheet
38	Session Summary
39	Clean Session Code - Part 1
40	Clean Session Code - Part 2

Session Overview

What Session 2 adds to the JavaScript fundamentals from Session 1.

This session moves from storing and reading values to working with those values. You will learn how operators form expressions, produce results, change variables, compare values, combine conditions and make compact decisions.

LEARNING ROADMAP

Operators
↓
Expressions
↓
Calculations
↓
Comparisons
↓
Logical Decisions
↓
Ternary Conditions
↓
Problem Solving

CORE VOCABULARY

Operator: a symbol or keyword that performs an operation. Operand: a value used by an operator. Expression: code that evaluates to a value.

Operators & Expressions

Build the language before using individual operators.

Operator, Operand, Expression

Start with the three words you will use throughout this session.

EXAMPLE

`10 + 5`

OPERATOR

`+` is the operator. It tells JavaScript to perform addition.

OPERANDS

10 and 5 are the operands. They are the values being operated on.

EXPRESSION

`10 + 5` is an expression because JavaScript evaluates it and produces a value: 15.

Arithmetic Operators

Symbols used to perform numerical operations.

Arithmetic Operators

Arithmetic operators let you calculate values. The source session also introduces increment and decrement as unary operators.

Operator	Name	Purpose	Example
+	Addition	Adds values	10 + 5
-	Subtraction	Subtracts values	10 - 5
*	Multiplication	Multiplies values	10 * 5
/	Division	Divides values	10 / 5
%	Modulus	Returns remainder	10 % 3
**	Exponentiation	Raises a value to a power	2 ** 3
++	Increment	Adds 1 to a variable	x++
--	Decrement	Subtracts 1 from a variable	x--

INTERVIEW NOTE

Arithmetic can involve unary operators such as ++ and --, and binary operators such as +, -, *, /.

Basic Arithmetic Examples

Use simple numbers first, then inspect the result.

Basic Arithmetic Examples

These are the direct calculations demonstrated in the source session.

SOURCE EXAMPLE

```
let a = 10;
let b = 5;

let res = a + b;
console.log("Addition result:", res);
```

Expected output

Addition result: 15

PRACTICE BLOCK

```
let x = 2;
let y = 4;

console.log("Addition:", x + y);
console.log("Subtraction:", x - y);
console.log("Multiplication:", x * y);
console.log("Division:", x / y);
console.log("Exponentiation:", x ** y);
```

Expected output

Addition: 6
Subtraction: -2
Multiplication: 8
Division: 0.5
Exponentiation: 16

Modulus Operator (%)

Returns the remainder after division.

Modulus

The modulus operator is especially useful when you need to test divisibility or identify even and odd numbers.

EXAMPLE

```
10 % 3
```

Expected output

1

HOW IT WORKS

10 divided by 3 produces a quotient of 3 with a remainder of 1. The % operator returns that remainder.

PRACTICAL USE

21 % 2 produces 1. Because the remainder is not 0, 21 is odd.

EVEN-NUMBER TEST

```
21 % 2 === 0
```

Pre-Increment vs Post-Increment

The placement of `++` changes the value produced by the expression.

Two Forms of Increment

Both forms increase a variable by one, but they differ in when the updated value is produced.

PRE-INCREMENT `++x`

Change first, then use the updated value.

`x = 2 → ++x → 3`

POST-INCREMENT `x++`

Use the current value first, then increase the variable.

`x = 2 → x++ → 2, then x = 3`

REMEMBER

The difference matters when `++x` or `x++` is part of a larger expression.

Pre-Increment (++x)

Change the variable first, then use the new value.

Pre-Increment

The increment happens before the expression produces its value.

EXAMPLE

```
let x = 2;  
console.log(++x);  
console.log(x);
```

EXECUTION

Initial $x = 2 \rightarrow ++x$ changes x to $3 \rightarrow$ the expression produces $3 \rightarrow x$ remains 3 after the statement.

Expected output

3
3

KEY TAKEAWAY

$++x$ means: increment first, then use.

Post-Increment (x++)

Use the current value first, then change the variable.

Post-Increment

The expression produces the old value; the variable is incremented as a side effect.

EXAMPLE

```
let y = 4;  
console.log(y++);  
console.log(y);
```

EXECUTION

Initial $y = 4 \rightarrow y++$ produces 4 $\rightarrow$ then y becomes 5.

Expected output

4
5

KEY TAKEAWAY

$x++$ means: use first, then increment.

Pre-Decrement vs Post-Decrement

The same timing rule applies when a variable is reduced by one.

Decrement Operators

Both `--x` and `x--` subtract 1, but the expression produces a different value depending on placement.

PRE-DECREMENT `--x`

Decrement first, then use the updated value.

`x = 5 → --x → 4`

POST-DECREMENT `x--`

Use the current value first, then decrement.

`x = 5 → x-- → 5, then x = 4`

SOURCE PATTERN

```
let x = 5;  
console.log(--x);  
console.log(x--);
```

REMEMBER

`--x` → change first. `x--` → use first.

Pre/Post Operator Execution Trace

Read the value produced and the variable state separately.

Execution Trace

Tracing makes increment and decrement expressions easier to predict.

TRACE EXAMPLE

```
let x = 3;  
  
console.log(++x, x++);  
console.log(x);
```

Step	Operation	x Before	Value Produced	x After
1	++x	3	4	4
2	x++	4	4	5

Expected output

```
4 4  
5
```

TRACE RULE

When you see ++ or --, write down two things: what value the expression produces and what value the variable has afterward.

Complex Compound Expressions

Break a long expression into individual operations.

Compound Increment Expression

The source session combines multiple post/pre operators in one expression. Read it one operator at a time.

EXPRESSION

```
x++ + y++ + ++y + y++
```

Expression	Value Used	Variable After
x++	5	x = 6
y++	5	y = 6
++y	7	y = 7
y++	7	y = 8

RESULT

$5 + 5 + 7 + 7 = 24$

Expected output

Compound Expression Result: 24
Value of y after expression execution: 8

EXAM TIP

Do not try to mentally evaluate a compound ++/-- expression in one jump. Track each operator in left-to-right order and record the variable state after each step.

Assignment Operators

Short forms that update and store a new value in the same variable.

Assignment Operators

Assignment operators combine an arithmetic operation with assignment.

Operator	Equivalent	Meaning
=	$x = \text{value}$	Assign
+=	$x = x + \text{value}$	Add and assign
-=	$x = x - \text{value}$	Subtract and assign
*=	$x = x * \text{value}$	Multiply and assign
/=	$x = x / \text{value}$	Divide and assign

CORE IDEA

`s += g` updates `s` by adding `g` to its current value, then stores the new value back into `s`.

EQUIVALENT FORMS

```
s = s + g;
```

```
// same effect
```

```
s += g;
```

INTERVIEW NOTE

Assignment operators are still expressions: the right-hand calculation happens first, then the updated value is stored in the left-hand variable.

Assignment Operators in Practice

Track the state of the variable after each shortcut.

Step-by-Step State Changes

The following example uses the same variables and sequence as the source session.

SOURCE SEQUENCE

```
let s = 30;
let g = 40;

s += g;
s -= g;
s *= g;
s /= g;
```

Step	Operation	s After
Start		30
1	s += g	70
2	s -= g	30
3	s *= g	1200
4	s /= g	30

Expected output

```
Addition Assignment: 70
Subtraction Assignment: 30
Multiplication Assignment: 1200
Division Assignment: 30
```

Comparison Operators

Operators that compare values and produce a Boolean result.

Comparison Operators

Comparison operators are used to form conditions. Their result is typically true or false.

Operator	Meaning
==	Loose equality
===	Strict equality
!=	Loose inequality
!==	Strict inequality
>	Greater than
<	Less than
>=	Greater than or equal
<=	Less than or equal

EXAMPLES

```
22 == 22
22 === 22
22 > 10
22 <= 22
```

KEY POINT

Comparison operators are especially useful inside conditions used by logical expressions and the ternary operator.

Loose vs Strict Equality

The distinction between `==` and `===` is an important interview concept.

Equality Operators

Use the source example with values that look the same but have different data types.

EXAMPLE

```
let d = 22;  
let e = "22";  
  
console.log(d == e);  
console.log(d === e);
```

== LOOSE EQUALITY

Allows type coercion in the comparison.
`22 == "22"` evaluates to true.

=== STRICT EQUALITY

Does not convert the operands before comparing. `22 === "22"` evaluates to false.

INTERVIEW REMINDER

`===` checks both value and type as part of the strict comparison.

Expected output

```
true  
false
```

Relational Operators

Compare whether one value is greater, smaller or equal in a directional sense.

Relational Comparisons

The relational operators from the source are `>`, `<`, `>=` and `<=`.

SOURCE EXAMPLE

```
let d = 22;  
let e = "22";  
  
console.log(d > e);  
console.log(d < e);  
console.log(d >= e);  
console.log(d <= e);
```

Expression	Result	Reason
<code>22 > "22"</code>	false	22 is not greater than 22
<code>22 < "22"</code>	false	22 is not less than 22
<code>22 >= "22"</code>	true	Values are equal
<code>22 <= "22"</code>	true	Values are equal

NOTE

Relational comparisons can involve type coercion when the operands have different types. Keep the result and the rule separate when explaining your answer.

Inequality Operators

Check whether values are different using loose or strict comparison.

Inequality

The source demonstrates the same type distinction for `!=` and `!==`.

EXAMPLE

```
console.log(10 != "10");  
console.log(10 !== "10");
```

!= LOOSE INEQUALITY

Allows coercion. Because 10 and "10" compare as equal after coercion, `10 != "10"` is false.

!== STRICT INEQUALITY

Checks without type conversion. Because the types differ, `10 !== "10"` is true.

REMEMBER

Equality and inequality come in loose and strict pairs: `==` / `!=` and `===` / `!==`.

Logical Operators

Combine Boolean conditions to build decisions.

Logical Operators

The three logical operators in this session are `&&`, `||` and `!`.

&&**AND**

Both conditions must be true.

||**OR**

At least one condition must be true.

!**NOT**

Reverses a Boolean result.

SOURCE EXAMPLE

```
let hasAge = true;
let hasID = false;

console.log(hasAge && hasID);
console.log(hasAge || hasID);
console.log(!hasID);
```

Expected output

```
false
true
true
```

Logical Truth Tables

A compact way to memorize the results of `&&`, `||` and `!`.

Truth Tables

Truth tables enumerate every Boolean combination for a logical operator.

A	B	A && B
true	true	true
true	false	false
false	true	false
false	false	false

A	B	A B
true	true	true
true	false	true
false	true	true
false	false	false

A	!A
true	false
false	true

MEMORY RULE

AND needs all true. OR needs at least one true. NOT simply flips the Boolean value.

Logical Operators in Practice

Translate a simple real-world rule into Boolean logic.

Practical Example

The source uses age and ID as two Boolean conditions.

EXAMPLE

```
let hasAge = true;
let hasID = false;

console.log(hasAge && hasID);
console.log(hasAge || hasID);
console.log(!hasID);
```

AND

`hasAge && hasID`

Result: false
Both must be true.

OR

`hasAge || hasID`

Result: true
At least one is true.

NOT

`!hasID`

Result: true
false becomes true.

REAL-WORLD PATTERN

Example: Can enter exam? A rule might require both a valid ID and valid registration, which maps naturally to `&&`.

Short-Circuit Evaluation

Logical operators can stop once the final outcome is already known.

Short-Circuit Evaluation

When JavaScript can already determine the result of a logical expression, it does not need to evaluate the remaining operand.

AND EXAMPLE

```
false && something
```

WHY?

The left side is already false. Because false && anything is false, the final result is known without needing the right side.

OR EXAMPLE

```
true || something
```

WHY?

The left side is already true. Because true || anything is true, the final result is known immediately.

BEGINNER NOTE

This session only needs the basic rule: logical evaluation can stop early when the final Boolean outcome is already determined.

Ternary Operator

A compact expression that chooses one of two values based on a condition.

Ternary Operator

The ternary operator is the JavaScript operator that takes three operands: a condition, a true result and a false result.

SYNTAX

```
condition ? valueIfTrue : valueIfFalse
```

SOURCE EXAMPLE

```
let age = 34;

let isEligible =
  age >= 18
    ? "You can Vote"
    : "You are not eligible for voting";
```

HOW IT WORKS

1. Evaluate the condition. 2. If true, produce the value after ?. 3. Otherwise, produce the value after :.

Ternary vs if-else

The same two-way decision can be expressed in different forms.

Choose a Form

Ternary is an expression that directly produces a value; if-else is a statement used to control program flow.

TERNARY

```
let result = age >= 18 ? "Eligible" : "Not Eligible";
```

IF-ELSE

```
let result;

if (age >= 18) {
  result = "Eligible";
} else {
  result = "Not Eligible";
}
```

PRACTICAL RULE

Use a ternary when the decision is short and produces one of two values. Avoid deeply nested ternaries because they become hard to read.

Practice 1 - Positive or Negative

Apply comparison + ternary.

Positive or Negative Number

Determine the sign of a number using a condition and the ternary operator.

SOLUTION FROM SOURCE

```
let c = -10;

const isPositive =
  c >= 0 ? "Positive" : "Negative";

console.log(isPositive);
```

LOGIC

Condition: $c \geq 0$. For -10, the condition is false, so the false branch is selected.

Expected output

Negative

Practice 2 - Find Largest Number

Apply comparison + ternary.

Find the Larger Value

Compare two numbers and select the larger one.

SOLUTION FROM SOURCE

```
let f = 10;  
let h = 20;  
  
const findLargest =  
  f > h ? "f is greater" : "h is greater";  
  
console.log(findLargest);
```

LOGIC

Condition: $f > h$. Because $10 > 20$ is false, the expression chooses "h is greater".

Expected output

h is greater

Practice 3 - Even or Odd

Use modulus + strict equality + ternary.

Even or Odd Number

The remainder after division by 2 is the key to the decision.

SOLUTION FROM SOURCE

```
let num = 21;

const isEven =
  num % 2 === 0 ? "even" : "odd";

console.log(isEven);
```

WHY % WORKS

$21 \% 2 = 1$. The remainder is not zero, so the equality check is false and the ternary selects "odd".

Expected output

odd

Additional Practice - Questions 4 to 8

New exercises based strictly on Session 2 concepts.

QUESTION 04 - Divisible by 5

Take a number and determine whether it is divisible by 5. Use % and a comparison.

QUESTION 05 - Smaller Number

Take two numbers and determine which is smaller using the ternary operator.

QUESTION 06 - Add and Assign

Use += to increase a variable by 25.

QUESTION 07 - Subtract and Assign

Use -= to decrease a variable by 10.

QUESTION 08 - Double the Value

Use *= to double the current value of a variable.

SKILL FOCUS

These questions practice modulus, comparisons, ternary expressions and compound assignment without introducing new syntax.

Additional Practice - Questions 9 to 13

New exercises focused on equality and logical conditions.

QUESTION 09 - Strict Equality

Check whether two values are strictly equal using `===`.

QUESTION 10 - Strict Inequality

Check whether two values are strictly different using `!==`.

QUESTION 11 - Secure Access

Given `hasPassword = true` and `isVerified = true`, use `&&` to determine whether the user can access the system.

QUESTION 12 - Admin Access

Given `isAdmin = false` and `isOwner = true`, use `||` to determine whether the user can access an admin feature.

QUESTION 13 - Adult or Minor

Use the ternary operator to classify an age as Adult or Minor.

SKILL FOCUS

These exercises move from direct comparison to combined Boolean logic and compact conditional expressions.

Challenge Questions

Predict results before running the code.

CHALLENGE 1

PREDICT

```
let x = 5;
console.log(x++);
console.log(++x);
console.log(x);
```

CHALLENGE 2

PREDICT

```
let a = 10;
let b = 10;
console.log(a == b);
console.log(a === b);
```

CHALLENGE 3

PREDICT

```
let x = 10;
x += 5;
x *= 2;
x -= 4;
console.log(x);
```

CHALLENGE 4

PREDICT

```
true && false || true
```

CHALLENGE 5

PREDICT

```
let age = 17;
let result = age >= 18 ? "Eligible" : "Not Eligible";
```

INSTRUCTION

Write your predicted output first. Then verify it by running the code. The goal is to explain why the result occurs, not just memorize the answer.

Answer Key - Questions 1 to 5

Solutions after the practice section.

QUESTION 1

Output: 4, 6, 6. Why: `x++` first produces 5 then increments to 6; `++x` then increments to 7? Wait: starting `x=5`, `x++` outputs 5 and `x=6`, `++x` outputs 7, final `x=7`.

QUESTION 2

Output: true, true. Both variables contain the same numeric value and the same type.

QUESTION 3

Output: 26. Start at 10 → `+=5` gives 15 → `*=2` gives 30 → `-=4` gives 26.

QUESTION 4

Output: true. AND is evaluated first: `true && false` gives false; `false || true` gives true.

QUESTION 5

Output: Not Eligible. `17 >= 18` is false, so the false branch is selected.

Answer Key - Questions 6 to 13

Solutions for the ten additional exercises.

QUESTION 06

Example: `let x = 10; x += 25; → x becomes 35.`

QUESTION 07

Example: `let x = 30; x -= 10; → x becomes 20.`

QUESTION 08

Example: `let x = 12; x *= 2; → x becomes 24.`

QUESTION 09

Use `a === b`. The result is true only when the value and type both match.

QUESTION 10

Use `a !== b`. It is true when the value or type differs under strict comparison.

QUESTION 11

Example: `hasPassword && isVerified → true` when both are true.

QUESTION 12

Example: `isAdmin || isOwner → true` when either condition is true.

QUESTION 13

Example: `age >= 18 ? "Adult" : "Minor".`

Challenge Answers

Reasoning matters more than the final output.

Challenge 1

Output: 5, 7, 7. Start $x=5 \rightarrow x++$ produces 5 and x becomes 6 $\rightarrow ++x$ makes $x=7$ and produces 7 $\rightarrow$ final $x=7$.

Challenge 2

Output: true, true. Both a and b are the number 10, so loose and strict equality both succeed.

Challenge 3

Output: 26. $10 + 5 = 15$; $15 \times 2 = 30$; $30 - 4 = 26$.

Challenge 4

Output: true. Evaluate AND first: $\text{true} \ \&\& \ \text{false} \rightarrow \text{false}$. Then $\text{false} \ || \ \text{true} \rightarrow \text{true}$.

Challenge 5

Output: Not Eligible. The condition $17 \geq 18$ is false.

Interview Revision - Core

Short, direct answers you can revise quickly.

What is an operator?

A symbol or keyword that performs an operation on one or more operands.

What is an operand?

A value used by an operator, such as 10 and 5 in `10 + 5`.

What is an expression?

Code that JavaScript evaluates to produce a value.

Difference between `==` and `===`?

`==` allows type coercion in many comparisons; `===` performs a strict comparison without converting the operands.

Difference between `!=` and `!==`?

`!=` is loose inequality; `!==` is strict inequality.

Pre-increment vs post-increment?

`++x` changes the variable before the value is produced; `x++` produces the current value first, then increments.

Interview Revision - Applied

Practical operator questions and concise answers.

What is the difference between += and +?

+ performs addition; += adds and stores the new value back into the variable.

What does % return?

The remainder left after division.

What does && do?

It requires both sides to be truthy for the logical result to be true and can short-circuit when the result is already known.

What does || do?

It produces a truthy result when at least one side is truthy and can short-circuit when the result is already known.

What does ! do?

It negates a Boolean result: true becomes false and false becomes true.

What is the ternary operator?

A compact conditional expression written as condition ? trueValue : falseValue.

Why is it called a conditional / ternary operator?

It is called conditional because it chooses a result based on a condition, and ternary because it has three operands.

Quick Revision Sheet

One-page cheat sheet for fast recall.

ARITHMETIC

+ - * / % **

INCREMENT / DECREMENT

++x → change first x++ → use first --x → change first x-- → use first

ASSIGNMENT

= += -= *= /=

COMPARISON

== === != !== > < >= <=

LOGICAL

&& || !

TERNARY

condition ? trueValue : falseValue

IMPORTANT REMINDER

== → loose equality

=== → strict equality

&& → all conditions must be true for a true AND result

|| → at least one condition must be true for a true OR result

Session Summary

What you should be able to do after Session 2.

Session Summary

Session 2 turns individual values into calculations, comparisons and decisions.

CONCEPTS COVERED

- Operators
- Operands
- Expressions
- Arithmetic operators
- Increment / Decrement
- Pre/Post behavior
- Assignment operators
- Comparison operators
- Equality / Inequality

SKILLS BUILT

- Relational operators
- Logical operators
- Truth tables
- Short-circuit evaluation
- Ternary operator
- Practical problem solving
- 10 additional practice questions
- Interview revision

WHAT YOU SHOULD BE ABLE TO DO

Perform arithmetic operations; trace pre/post increment and decrement; use compound assignment; compare values correctly; explain `==` vs `===`; build Boolean conditions; understand basic short-circuit evaluation; write ternary expressions; solve simple conditional problems.

Clean Session Code - Part 1

Reference version of the Session 2 source material, formatted consistently.

PART 1

```
console.log("=== SESSION 2: OPERATORS & LOGIC ===");

// Arithmetic
let a = 10;
let b = 5;
let res = a + b;
console.log("Addition result variable:", res);

let x = 2;
let y = 4;
console.log("Addition (+):", x + y);
console.log("Subtraction (-):", x - y);
console.log("Multiplication (*)", x * y);
console.log("Division (/):", x / y);
console.log("Exponentiation (**):", x ** y);

// Increment / Decrement
console.log("Pre-Increment (++x):", ++x);
console.log("Post-Increment (y++):", y++);
console.log("Value of y after post-increment:", y);
console.log(++x, x++);
console.log("Value of x after pre & post increment:", x);
console.log("Compound Expression Result:", x++ + y++ + ++y + y++);
console.log("Value of y after expression execution:", y);
console.log("Pre-Decrement (--x):", --x);
console.log("Post-Decrement (x--):", x--);
```

Clean Session Code - Part 2

Continuation of the formatted Session 2 source material.

PART 2

```
// Assignment
let s = 30;
let g = 40;
s += g;
console.log("Addition Assignment:", s);
s -= g;
console.log("Subtraction Assignment:", s);
s *= g;
console.log("Multiplication Assignment:", s);
s /= g;
console.log("Division Assignment:", s);

// Comparison
let d = 22;
let e = "22";
console.log("Loose Equality (d == e):", d == e);
console.log("Strict Equality (d === e):", d === e);
console.log("Greater Than (d > e):", d > e);
console.log("Less Than (d < e):", d < e);
console.log("Greater Than or Equal (d >= e):", d >= e);
console.log("Less Than or Equal (d <= e):", d <= e);
console.log("Abstract Inequality (10 != '10'):", 10 != "10");
console.log("Strict Inequality (10 !== '10'):", 10 !== "10");

// Logical
let hasAge = true;
let hasID = false;
console.log("Logical AND (&&):", hasAge && hasID);
console.log("Logical OR (||):", hasAge || hasID);
console.log("Logical NOT (!):", !hasID);

// Ternary + Practice
let age = 34;
let isEligible = age >= 18 ? "You can Vote" : "You are not eligible for voting";
console.log("Eligibility:", isEligible);

let c = -10;
const isPositive = c >= 0 ? "Positive" : "Negative";
console.log("Is Positive:", isPositive);

let f = 10;
let h = 20;
const findLargest = f > h ? "f is greater" : "h is greater";
console.log("Largest:", findLargest);

let num = 21;
const isEven = num % 2 === 0 ? "even" : "odd";
console.log("Is Even:", isEven);
```